

TD Nr 11 – Clause SELECT (Suite)

3. Les capacités de calcul et de transformation (suite)

- **Fonctions scalaires** : Transformer du texte en majuscules (UPPER()), extraire une partie d'une date ou arrondir des nombres.

Exemple : Pour écrire en majuscule les noms des villes. Voyez la différence !

select nom_client, ville_client from client ;

	nom_client	ville_client
☐ Éditer ☐ Copier ☐ Supprimer	Martin Jean	Paris
☐ Éditer ☐ Copier ☐ Supprimer	Bernard Marie	Lyon
☐ Éditer ☐ Copier ☐ Supprimer	Dubois Thomas	Marseille
☐ Éditer ☐ Copier ☐ Supprimer	Thomas Alice	Lille
☐ Éditer ☐ Copier ☐ Supprimer	Robert Julien	Bordeaux
☐ Éditer ☐ Copier ☐ Supprimer	Richard Sophie	Toulouse
☐ Éditer ☐ Copier ☐ Supprimer	Petit Nicolas	Bruxelles
☐ Éditer ☐ Copier ☐ Supprimer	Durand Claire	Genève
☐ Éditer ☐ Copier ☐ Supprimer	Leroy Antoine	Nice
☐ Éditer ☐ Copier ☐ Supprimer	Moreau Lucie	Rennes

select nom_client, upper(ville_client) from client ;

	nom_client	upper(ville_client)
☐ Éditer ☐ Copier ☐ Supprimer	Jean	PARIS
☐ Éditer ☐ Copier ☐ Supprimer	Bernard Marie	LYON
☐ Éditer ☐ Copier ☐ Supprimer	Dubois Thomas	MARSEILLE
☐ Éditer ☐ Copier ☐ Supprimer	Thomas Alice	LILLE
☐ Éditer ☐ Copier ☐ Supprimer	Robert Julien	BORDEAUX
☐ Éditer ☐ Copier ☐ Supprimer	Richard Sophie	TOULOUSE
☐ Éditer ☐ Copier ☐ Supprimer	Petit Nicolas	BRUXELLES
☐ Éditer ☐ Copier ☐ Supprimer	Durand Claire	GENÈVE
☐ Éditer ☐ Copier ☐ Supprimer	Leroy Antoine	NICE
☐ Éditer ☐ Copier ☐ Supprimer	Moreau Lucie	RENNES

Exemple : pour extraire une partie d'une date, on utilise la fonction EXTRACT. Dans la BD *banque*, nous n'avons pas de colonne contenant des dates, aussi utilisons pour cette fois-ci la table commande de la BD *commerce*.

```
SELECT EXTRACT(YEAR FROM date_commande) AS annee,  
       EXTRACT(MONTH FROM date_commande) AS mois,  
       EXTRACT(DAY FROM date_commande) AS jour  
FROM Commandes;
```

année	mois	jour
2026	2	10
2026	2	11

Exemple : On utilise les fonctions ROUND() ou CEIL() pour arrondir vers le haut et FLOOR() pour arrondir vers le bas. Veuillez noter vous-même la différence entre ROUND() et CEIL().

```
-- Arrondi à l'entier le plus proche  
SELECT ROUND(15.7) AS resultat; -- Retourne 16  
SELECT ROUND(15.4) AS resultat; -- Retourne 15  
  
-- Arrondi avec un nombre précis de décimales  
SELECT ROUND(salaire_mensuel, 2) AS salaire_propre  
FROM Employes;  
-- Si le salaire est 2500.4587, il devient 2500.46
```

```
SELECT CEIL(12.1) AS resultat; -- Retourne 13
```

```
SELECT FLOOR(12.9) AS resultat; -- Retourne 12
```

Notez que dans les deux premiers et les deux derniers exemples, la clause FROM est omise car il ne s'agit pas de requêtes à une BD mais de simples calculs comme avec une calculatrice.

- **Logique conditionnelle (CASE WHEN)** : Vous pouvez créer des colonnes "intelligentes" qui affichent une valeur selon une condition (un genre de "Si... Alors" directement dans la requête).

```

SELECT
    num_pret,
    CASE
        WHEN montant < 10000 THEN 'Petits emprunteurs'
        WHEN montant < 30000 and montant >= 10000 THEN 'Moyens emprunteurs'
        ELSE 'Gros emprunteurs'
    END AS categorie
FROM pret;

```

Le tableau à gauche sert juste à vous aider à vérifier le résultat affiché dans le tableau à droite. Notez aussi que la colonne catégorie n'existe pas dans la table originale mais a été *ajoutée* par la requête.

	num_pret	montant	code_filiale
☐ Éditer ☐ Copier ☐ Supprimer	P001	150000.00	1
☐ Éditer ☐ Copier ☐ Supprimer	P002	5000.00	2
☐ Éditer ☐ Copier ☐ Supprimer	P003	25000.00	3
☐ Éditer ☐ Copier ☐ Supprimer	P004	300000.00	4
☐ Éditer ☐ Copier ☐ Supprimer	P005	1200.00	5
☐ Éditer ☐ Copier ☐ Supprimer	P006	45000.00	6
☐ Éditer ☐ Copier ☐ Supprimer	P007	8000.00	7
☐ Éditer ☐ Copier ☐ Supprimer	P008	120000.00	8
☐ Éditer ☐ Copier ☐ Supprimer	P009	2500.00	9
☐ Éditer ☐ Copier ☐ Supprimer	P010	18000.00	10

	num_pret	categorie
☐ Éditer ☐ Copier ☐ Supprimer	P001	Gros emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P002	Petits emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P003	Moyens emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P004	Gros emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P005	Petits emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P006	Gros emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P007	Petits emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P008	Gros emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P009	Petits emprunteurs
☐ Éditer ☐ Copier ☐ Supprimer	P010	Moyens emprunteurs

4. Les fonctions d'agrégation

Couplé à un GROUP BY, le SELECT devient un outil d'analyse statistique puissant grâce à des fonctions comme :

- COUNT() : Compter le nombre d'entrées.
- SUM() : Faire la somme d'une colonne numérique.
- AVG() : Calculer une moyenne.
- MIN() / MAX() : Trouver les valeurs extrêmes.

Les fonctions d'agrégation sont des outils puissants qui permettent de calculer une valeur unique à partir d'un ensemble de lignes. La grande différence réside dans l'utilisation ou non du GROUP BY.

1. Fonctions d'agrégation SANS GROUP BY

Lorsqu'on n'utilise pas de regroupement, SQL traite **toute la table comme un seul bloc**. Le résultat ne contient alors qu'une **seule ligne**.

Exemples :

Imaginons une table Ventes avec les colonnes montant et quantité.

- **COUNT** : Compter le nombre total de lignes.

```
SELECT COUNT(*) AS total_ventes FROM Ventes;
-- Résultat : 150 (Il y a eu 150 transactions au total)
```

- **SUM** : Faire la somme globale.

```
SELECT SUM(montant) AS chiffre_affaires FROM Ventes;
-- Résultat : 45000 (Le cumul de toutes les ventes)
```

- **AVG, MIN, MAX** : Statistiques générales.

```
SELECT AVG(montant) AS panier_moyen,
       MIN(montant) AS vente_la_plus_basse,
       MAX(montant) AS vente_la_plus_haute
FROM Ventes;
```

2. Fonctions d'agrégation AVEC GROUP BY

Ici, on demande à SQL de découper la table en "groupes" (par exemple par catégorie, par vendeur ou par mois) et d'appliquer le calcul sur chaque groupe séparément.

Exemples :

- **Répartition par catégorie :**

```
SELECT categorie,
       COUNT(*) AS nombre_produits,
       SUM(montant) AS total_par_categorie
FROM Ventes
GROUP BY categorie;
```

Ici, si tu as 5 catégories, tu obtiendras 5 lignes de résultats.

- **Performance par vendeur :**

```
SELECT vendeur_id,
       AVG(montant) AS moyenne_vente_vendeur
FROM Ventes
GROUP BY vendeur_id;
```

3. Comparaison : Les différences clés

Caractéristique	Sans GROUP BY	Avec GROUP BY
Périmètre	Toute la table.	Sous-ensembles de la table.
Nombre de lignes en sortie	Toujours 1 seule ligne .	Autant de lignes que de groupes.
Colonnes dans le SELECT	Uniquement des agrégats (SUM, AVG...).	Les colonnes de groupe + les agrégats.
Utilisation type	Obtenir un total général.	Créer un rapport ou un tableau croisé.

Le piège à éviter : "L'oubli du Group By"

Si tu tentes de sélectionner une colonne "normale" à côté d'une fonction d'agrégation sans faire de regroupement, SQL te renverra une erreur :

```
-- ❌ ERREUR (La plupart des systèmes SQL comme PostgreSQL)
SELECT nom_produit, SUM(montant) FROM Ventes;

-- ✅ CORRECT
SELECT nom_produit, SUM(montant) FROM Ventes GROUP BY nom_produit;
```

La clause **HAVING** est souvent confondue avec le WHERE, mais elle a un rôle bien précis : elle sert à filtrer les **résultats d'une agrégation**.

Pour faire simple :

- **WHERE** filtre les **lignes individuelles** (avant le calcul).
- **HAVING** filtre les **groupes** (après le calcul).

1. Exemple : Filtrer des groupes de ventes

Imaginons que tu veuilles la liste des départements qui ont réalisé un chiffre d'affaires supérieur à 50 000 €. Tu ne peux pas utiliser WHERE car le total n'existe pas encore tant que le regroupement n'est pas fait.

```
SELECT departement, SUM(montant) AS total_ventes
FROM Ventes
GROUP BY departement
HAVING SUM(montant) > 50000;
```

2. Cumuler WHERE et HAVING

C'est ici que tu vois la puissance du SQL. Tu peux filtrer les données de base, faire le calcul, puis filtrer le résultat.

Exemple : On veut les vendeurs ayant fait plus de 10 ventes, mais on ne compte que les ventes de l'année 2025.

```
SELECT vendeur_id, COUNT(*) AS nb_ventes
FROM Ventes
WHERE date_vente >= '2025-01-01' -- 1. Filtre les lignes d'abord
GROUP BY vendeur_id              -- 2. Regroupe par vendeur
HAVING COUNT(*) > 10;            -- 3. Filtre les vendeurs "performants"
```

3. Comparaison : WHERE vs HAVING

Caractéristique	WHERE	HAVING
Moment d'action	Agit sur les lignes brutes de la table.	Agit sur les groupes créés par <code>GROUP BY</code> .
Fonctions d'agrégation	Interdit (ex: <code>WHERE SUM(x) > 0</code> ne marche pas).	Autorisé (ex: <code>HAVING SUM(x) > 0</code>).
Objectif	Éliminer les données inutiles avant calcul.	Éliminer les résultats qui ne remplissent pas un quota.

Le mémo de l'ordre d'exécution

Pour ne plus jamais te tromper, retiens l'ordre dans lequel SQL traite ta requête :

1. `FROM` (D'où viennent les données ?)
2. `WHERE` (Quelles lignes on garde ?)
3. `GROUP BY` (Comment on les regroupe ?)
4. `HAVING` (Quels groupes on garde ?)
5. `SELECT` (Qu'est-ce qu'on affiche ?)
6. `ORDER BY` (Dans quel ordre on trie le résultat ?)

5. L'ordre d'exécution (Le piège classique)

C'est la particularité la plus contre-intuitive : bien que SELECT soit écrit en **premier** dans votre requête, il est l'un des **derniers** à être exécuté par le moteur de la base de données (après le FROM, le JOIN, le WHERE et le GROUP BY).

C'est pour cette raison que vous ne pouvez pas utiliser un alias créé dans le SELECT à l'intérieur d'une clause WHERE.

Résumé des composants

Composant	Utilité
*	Récupère tout.
DISTINCT	Supprime les doublons.
AS	Renomme une colonne (alias).
CASE	Ajoute de la logique conditionnelle.